

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ENGINEERING FUNDAMENTALS

Software Classes

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.1

Business

Custom Software Development — CSD

Commercial development of **non**-standardised, **fully** individualised, and **non**-reusable **company**-specific software for a **single** customer.

Standard Software Development — STD

Commercial development of standardised, **partially** customisable, and **fully** reusable **domain**-specific software for **many** customers.

Open Source Software Development — OSS

Non-commercial development of standardised, **highly** customisable, and **fully** reusable **generic** software for **many** customers.

Class: Graphics & Media

target audience: **consumers & enterprises**

Graphics Editing Application — GEA

Software for editing and rendering graphics in vector and bitmap format.

Examples: Cinema4D, Maya, Blender, After Effects, Illustrator, Inkscape, Scribus, Photoshop, GIMP, etc.

~~CSD~~ STD OSS

Graphics Animation Engine — GAE

Software for animating the 2D/3D virtual worlds of games and overlays of TV productions.

Examples: Unity, Unreal Engine, CryENGINE, Godot, HUDS, SPX-GC, Holographics, H2R Graphics, etc.

CSD STD ~~OSS~~

Audio/Video-Processing System — AVS

Software for live-processing and post-production of audio/video based multimedia streams.

Examples: vMix, OBS Studio, VLC, Lossless Cut, Handbrake, Adobe Premiere, FFmpeg, Nimble, etc.

~~CSD~~ ~~STD~~ OSS

Class: Business & Data

target audience: **consumers & enterprises**

Office Productivity Application — OPA

Software for productivity in the desktop-based office environment.

Examples: PowerPoint, Excel, Word, Visio, OmniGraffle, LibreOffice, Outlook, XMind, Firefox, Chrome, etc.

~~CSD~~ STD OSS

Business Information System — BIS

Software for driving business processes through interactive information management.

Examples: Vote, CampS, Mission Control, IPW, KEZ-PSC, TimeSheet, SAP ERP, OpenProject, etc.

CSD STD ~~OSS~~

Data Management System — DMS

Software for protocol-based storing and retrieving of persistent data.

Examples: NextCloud, PostgreSQL, CockroachDB, Redis, InfluxDB, Neo4J, Tendermind, Gitea, Vault, etc.

~~CSD~~ ~~STD~~ OSS

Class: Machinery & Network

target audience: **consumers & enterprises**

Technical Control System — TCS

Software for controlling a physical machinery or technical system.

Examples: AquaTherm, AVM! FritzBox Firmware, BirdDog Camera Firmware, etc.

CSD ~~STD~~ ~~OSS~~

Network Communication System — NCS

Software for protocol-based communication of data over a computer network.

Examples: Apache, NGINX, HAProxy, Mosquitto, RabbitMQ, Node-RED, KeyCloak, etc.

~~CSD~~ ~~STD~~ OSS

Operating System Kernel — OSK

Software kernel for low-level operating a physical or virtual device and run programs on it.

Examples: Windows, macOS, iOS, Linux, FreeBSD, QNX, ChibiOS/RT, Kubernetes, Wildfly, etc.

~~CSD~~ STD OSS

Class: Development & Tools

target audience: **vendors & suppliers**

Software Development Kit — SDK

Software libraries and frameworks of reusable functionality for developing software.

Examples: NDI SDK, HAPI, GraphQL-IO, Sequelize, JDK, Spring, Hibernate, etc.

~~CSD~~ ~~STD~~ OSS

Software Development Tools — SDT

Software tools for editing, linting, compiling, packaging, distributing, and installing software.

Examples: Visual Studio Code, Sublime Text, GCC, GNU Binutils, NPM, JDK, Docker, Helm, etc.

~~CSD~~ ~~STD~~ OSS

Operating System Tools — OST

Software tools for high-level operating a physical or virtual computing device.

Examples: Coreutils, Bash, Vim, TMux, FZF, cURL, RSYNC, OpenSSH, etc.

~~CSD~~ ~~STD~~ OSS

audience & deliverable

Intellectual Content: Version 1.1.1 (2022-12-10), Authored 2021-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2022-10-29), Copyright © 2021-2022 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ENGINEERING FUNDAMENTALS

Software Development Approaches

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.2

Development Approaches

Software Prototyping *mocking* SP

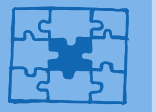
Develop an early sample or model of a software solution by mocking and cheating in order to just once test a concept, idea or process.



Example: Customer Sales Demo

Software Bricolage *integrating* SB

Develop a single instance of a software solution by tinkering, cobbling and integrating partial solutions in order to prove feasibility or just provide a service.



Example: Company-Internal SaaS

Software Craftsmanship *crafting* SC

Develop a production-grade software solution by professional, clean but plain craftsmanship means in order to solve a usually complicated problem.



Example: Open Source Framework

Software Engineering *teaming* SE

Develop a production-grade software solution by a professional, risk-hedged engineering approach in order to solve a usually complex problem.



Example: Business Information System

Continuum & Process

The four development approaches do *not* form a hierarchy, but can be combined in practice: **Prototyping** and **Bricolage** can be earlier stages of **Craftsmanship** or **Engineering**. **Craftsmanship** can be part of **Bricolage** or **Engineering**. Each approach requires a special skill (mocking, integrating, crafting, teaming).

Development Approaches: **Characteristics Comparison** *

	Effort: Person-Days	Effort: Persons	Process: Risk-Hedge	Process: Traceability	Solution: Target Technology	Solution: Production-Grade	Solution: Sustainability	Solution: Claim	Solution: Life-Time Months	Solution: Lines of Code (k)
Software Prototyping	1-20	1-2	-	-	-	-	-	5%	0-3	0-3
Software Bricolage	5-100	1-2	-	-	X	(X)	-	60%	3-24	1-10
Software Craftsmanship	5-100	1-2	-	-	X	X	X	100%	24-48	5-25
Software Engineering	>150	5-50	X	X	X	X	X	80%	>48	>25

* All figures are just rough orders of magnitude for indication and illustration purposes.

Key Message

All four approaches are equally essential in practice. Which one(s) to choose, entirely depends on the particular requirements.

Development Approaches: **Success Patterns**

	Software Prototyping	**Software Bricolage**	**Software Craftsmanship**	**Software Engineering**
Performance	One-Man-Show	One-Man-Show	One-Man-Show	**Team Play**
Responsibility	Single	Single	Single	**Separated**
Model	Mental	Mental	Mental/**Documented**	**Documented**
Decisions	Implicit	Implicit	Implicit/**Explicit**	**Explicit**
Process	**Minimized**	**Partial**	**Partial**	Complete
Optimisation	**Time**	**Efficiency**	**Effectiveness**	**Economics**
Risks	Ignore	Ignore	Ignore	**Mitigate**
Stakeholders	Ignore	Ignore	Ignore	**Manage**
Mastering	**Time-Constraint**	Complexity	**Complication**	Complexity
Solutions	**Use Full**	Use Partial	Use Partial	Use Partial
Standards	Use	Use	**Potentially Create**	Use
Efforts	**Configuration**	**Integration**	Programming	Programming
Target	**Demo**	Solution	**Product**	**Product**
Sustainability	**No**	**Partial**	Full	Full
Traceability	**No**	**No**	**Partial**	Full

Intellectual Content: Version 1.04 (2022-11-10), Authored 2021-2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.03 (2021-10-25), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

goal & approach

ENGINEERING FUNDAMENTALS

Software Engineering

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.3

Engineering

Machinery

The collection of physical components of a mechanical apparatus using electrical power to perform a certain task.

created by

Engineering

The systematic application of **scientific and technological knowledge, principles, approaches, practices, methods, and experiences**, to **design, build, operate, and maintain** complex production-grade solutions through a **traceable** process. *

taking into account

Engineering Code of Conduct

Commit yourself to the **highest ethical and professional conduct**, accept responsibility in making decisions consistent with the **safety, health, and welfare of the public**, apply the **state of the art in science and technology** of your field, perform services only in areas of your **competence**, and comply with the principles of **sustainable development**. *

* derived from definitions and Code of Ethics statements of ASCE, NSPE, ACM, IEEE, etc.

specialised by

Hardware

The collection of physical components (machinery) of a computer system.

operated by

Software Engineering

Software

The collection of instructions and data that tell a computer system how to work.

created by

Software Engineering (transferred)

The systematic application of scientific and technological knowledge, principles, approaches, practices, methods, and experiences, to design, build, operate, and maintain complex production-grade **software** solutions through a traceable process.

transferred to

simplified to

Software Engineering (simplified)

The systematic application of **engineering** knowledge, principles, approaches, practices, methods, and experiences to **the development of software.**

also taking into account

Wording

Engineering: derived from the *Latin* words **ingeniare** (to create, generate, contrive, devise) and **ingenium** (cleverness).

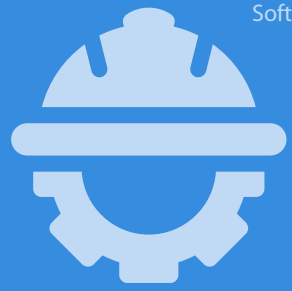
Intellectual Content: Version 1.02 (2021-09-06) Authored 2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.02 (2021-09-11) Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

profession & ethics

ENGINEERING FUNDAMENTALS

Profession Characteristics

TECHNISCHE UNIVERSITÄT MÜNCHEN

	creative	rational
	involving the use of **strong imagination** and **original ideas**	involving the use of **analytical thinking** and **logical reasoning**
practical involving the **actual doing** and **real application** of something	**CRAFTSMAN** Handwerker Objectives: **solve problems** **create uniques** … Software Engineering: **Programming** **Configuration** … 	**ENGINEER** Ingenieur Objectives: **solve problems** **create solutions** … Software Engineering: **Estimation** **Automation** … 
theoretical involving the **manifested imagination** or **analytical study** of something	**ARTIST** Künstler Objectives: **create beauty** **create uniques** … Software Engineering: **Visual Design** **Coding Style** … 	**SCIENTIST** Wissenschaftler Objectives: **analyse problems** **create knowledge** … Software Engineering: **Methodology** **User Research** … 

INVENTOR Erfinder



EF 01.4

Intellectual Content: Version 1.0.0 (2024-06-10), Authored 2024 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2024-06-10), Copyright © 2024 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts on



ENGINEERING FUNDAMENTALS

TRUE Manifesto

true-manifesto.org

TECHNISCHE UNIVERSITÄT MÜNCHEN



targeted

adequate
suitable
focused

Statement: **We focus on adequate and suitable solutions and approaches.**

Rationale: **Both solutions and approaches have to be in a reasonable proportion to the problem.**

Implications:
- We avoid both over-engineered and cobbled-together solutions.
- We avoid "one-size-fits-all" approaches.
- We suitably adapt solutions, tools and methods.



reasoned

considered
assessed
deliberate

Statement: **We think carefully and holistically in advance about our solutions and approaches.**

Rationale: **We always think large, even if we have to act small, because thinking in advance is more efficient and effective than correcting afterwards.**

Implications:
- We always develop the "big picture" first and add ancillary details as late as possible.
- We are opinionated and steadfast regarding our decisions and solutions.
- We know that conceptual modeling is key to understanding both problems and solutions.



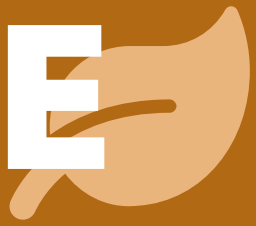
up-to-date

educated
experienced
insistent

Statement: **We develop high-quality solutions on the basis of up-to-date methods and technologies.**

Rationale: **We have to cope with the fact that the IT world is recurrently revolutionizing itself.**

Implications:
- We continuously educate ourselves.
- We continuously and critically challenge and assess emerging approaches and products.
- We are not satisfied with mediocre solutions.



evolutionary

sustainable
harmonic
contextual

Statement: **We develop sustainable solutions that optimally fit into their context.**

Rationale: **Nature teaches us that only evolutionary approaches and solutions have a good chance to survive in the long run.**

Implications:
- We actively learn from experiences of the past in order to improve the future.
- We avoid "quick hacks", as they are not long-term solutions, but just short-term means to get rid of problems.
- We assure that our solutions can be reasonably maintained in the long-term.

EF 01.5

entitlement & values

Intellectual Content: Version 1.0.7 (2021-10-25), Authored 2019-2021 by Dr. Ralf S. Engelschall, Christian Reiber, Rudolf Koster and Matthias Brusdeylins
Graphical Illustration: Version 1.0.7 (2021-10-25), Copyright © 2019-2021 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ENGINEERING FUNDAMENTALS

Discipline Claim

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.6

FA FARSIGHTED weitblickend

Be farsighted in your solution finding.

Sei weitblickend in deiner Lösungsfindung.

AR: Scalable Hub'n'Spoke
DV: Plugin SPI



TE TENET-ORIENTED grundsatzorientiert

Orientate yourself on fixed tenets in your approach and solution finding.

Orientiere dich an festen Grundsätzen in deinem Vorgehen und deiner Lösungsfindung.

AR: Separation of Concern
DV: Strict Coding-Style



TH THOUGHTFUL wohlüberlegt

Act thoughtful in your approach and solution finding.

Agiere wohlüberlegt in deinem Vorgehen und deiner Lösungsfindung.

AR: Modularization
DV: Algorithmical Control Structure



HO HOLISTICALLY ganzheitlich

Think holistically and in the long-term when finding your solutions.

Denke ganzheitlich und langfristig in deiner Lösungsfindung.

AR: Walking Skeleton Design
DV: Consistent Error Handling



AD ADEQUATE angemessen

Ensure that your approach and solutions are adequate to the boundary conditions.

Sorge dafür, daß dein Vorgehen und deine Lösungen angemessen zu den Rahmenbedingungen sind.

AR: No Cloud-Native Complexity
DV: No Over-Engineered Abstractions



FE FEASIBLE machbar

Ensure that your approach and solutions can be realised at reasonable costs.

Sorge dafür, daß dein Vorgehen und deine Lösungen mit vernünftigen Kosten realisiert werden können.

AR: Existing Framework Functionality
DV: Realistic Programming Model

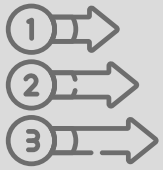


IN INCREMENTAL inkrementell

Apply the depth of your discipline incrementally.

Wende die Tiefe deiner Disziplin inkrementell an.

AR: Identified Solution Cruxes
DV: Minimum Viable Product



VA VALUEABLE wertvoll

Provide clearly recognizable added values with your approach and solutions.

Liefere klar ersichtliche Mehrwerte mit deinem Vorgehen und deinen Lösungen.

AR: Technology Stack Design
DV: User-Story-Driven Functionality



SU SUSTAINABLE nachhaltig

Create sustainable solutions that are well integrated into their environment.

Erschaffe nachhaltige Lösungen, die gut in ihre Umgebung integriert sind.

AR: Interoperable Interfaces
DV: Maintainable Code



Intellectual Content: Version 1.0.1 (2024-06-17) Authored 2023-2024 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.1 (2024-06-17) Copyright © 2023-2024 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ENGINEERING FUNDAMENTALS

Sustainability

TECHNISCHE UNIVERSITÄT MÜNCHEN

EF 01.7

HI Minimize HARDWARE Idleness

Minimize the idleness and maximize the utilization of existing hardware resources.

Rationale: Unused or under-utilized hardware are an unnecessary waste of already available resources.

Keywords: Virtualization, Utilization.



DE Minimize DESIGN Excessiveness

Minimize the excessiveness and maximize the adequacy of solution designs.

Rationale: Non-adequate designs cause unnecessary complexity and waste resources.

Keywords: Reduced Libraries, Immutability.



HE Minimize HUMAN Effort

Minimize the efforts of humans and maximize the efforts of machines in all production and operation processes.

Rationale: Delegating tasks to machines gives humans the possibility to concentrate on more important tasks.

Keywords: Computer, Robot, Automation.



SI Minimize SOFTWARE Inefficiency

Minimize the inefficiency and maximize the efficiency of software applications and their development processes.

Rationale: Efficient software and development processes consume less resources.

Keywords: Caching, Monolith.



SE Minimize SOLUTION Ephemerality

Minimize the ephemerality and maximize the life-span of any type of solutions.

Rationale: Short life-spans of solutions cause unnecessary short renewals and this way wastes resources.

Keywords: High Quality, Best Practice.



EC Minimize ENERGY Consumption

Minimize the consumption and maximize the saving of energy in all production and operation processes.

Rationale: Electric energy still has to be partially generated from non-renewable resources.

Keywords: Eco Mode, Reduced CI/CD.



IA Minimize INFORMATION Amount

Minimize the total amount of gathered, transmitted, stored and spreaded information.

Rationale: Reduced amount of information means less data transmission, less data storage, less GDPR issues, etc.

Keywords: Compression, No Big Data.



EE Minimize ECOSYSTEM Exploitation

Minimize the exploitation and maximize the back-contribution in any type of ecosystems.

Rationale: The consumer and provider behaviour have to be in balance for every long-lasting ecosystem.

Keywords: Open Source Software.



CE Minimize CARBON Emission

Minimize the carbon emission and hence the footprint during any type of production and operation processes.

Rationale: Climate change and global warming is partially caused or at least accelerated by carbon emissions.

Keywords: Reduced CO2 Footprint.



Intellectual Content: Version 1.00 (2022-10-31) Authored 2022 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.00 (2022-10-31) Copyright © 2022 Dr. Ralf S. Engelschall <http://engelschall.com> All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.